



TRABAJO INTEGRADOR

Del Perceptrón a las CNN:

Construyendo un pipeline completo de Deep Learning para clasificación de imágenes médicas

Semanas	1 a 7 — Trabajo integrador final del primer bloque
Modalidad	Grupal (3 a 3)4 integrantes)
Entregable	Un único notebook Jupyter ejecutado (.ipynb con todos los outputs visibles)
Nombre archivo	deep_learning_integrador_[apellidos].ipynb
Plazo	20 mayo 2026 – 23.59
Puntaje total	20 puntos
Framework	PyTorch o TensorFlow/Keras (a elección del grupo)
Plataforma	Google Colab (GPU gratuita recomendada)

1. Contexto y Objetivo General

Durante las primeras siete semanas del curso se han construido, capa a capa, los fundamentos teóricos y prácticos del Deep Learning: desde la formulación matemática del aprendizaje automático hasta las arquitecturas convolucionales modernas. Este trabajo integrador tiene como objetivo demostrar que los estudiantes son capaces de conectar todos esos conceptos en un único pipeline coherente y funcional.

El hilo conductor del trabajo es un problema real de clasificación de imágenes médicas. A lo largo del notebook, cada sección corresponde a los temas de una semana específica, aplicados progresivamente sobre el mismo dataset y problema.

Al finalizar el trabajo, el grupo habrá construido, evaluado y analizado críticamente un sistema completo que va desde el análisis exploratorio de datos hasta una CNN con Transfer Learning, pasando por regresión logística, redes neuronales completamente conectadas, regularización y optimización.



2. Dataset

Chest X-Ray Images (Pneumonia) — Kaggle

El trabajo utiliza un dataset público de radiografías de tórax con dos clases: pulmón normal y pulmón con neumonía.

Repositorio	https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia
Clases	2: NORMAL, PNEUMONIA
Imágenes	~5,800 imágenes en escala de grises (variable resolución)
Split	Train / Validation / Test ya provisto por el dataset
Licencia	CC BY 4.0

Justificación: el dataset es binario (ideal para semanas 1–3), de tamaño manejable en Colab, con imágenes reales que requieren preprocesamiento (semana 1), se presta a regresión logística como baseline (semana 2), entrenamiento de MLP (semana 3), regularización (semana 4), y CNN (semanas 5–7). Tiene impacto social real, lo que enriquece la discusión crítica.

3. Estructura del Notebook

El notebook debe entregarse con la siguiente estructura de secciones Markdown. Cada sección lleva el número de semana que le corresponde para facilitar la corrección.

SEMANA 1	Fundamentos del aprendizaje automático y análisis exploratorio
---------------------------	---

Temas que cubre: historia y evolución de la IA, tipos de aprendizaje, representación de datos, formulación matemática.

En esta sección el grupo debe:

- Describir brevemente el problema: ¿qué tipo de aprendizaje es? ¿supervisado, no supervisado o por refuerzo? Justificar.
- Formular matemáticamente el problema: definir el espacio de entrada X , la salida Y , la hipótesis h , y la función de pérdida L que se minimizará a lo largo del trabajo.
- Realizar un análisis exploratorio del dataset (EDA):
 - Distribución de clases (¿está balanceado?)
 - Visualizar al menos 10 imágenes de cada clase



- Histograma de valores de píxel por clase
- Estadísticas básicas: media, desviación estándar por clase
- Implementar el preprocesamiento de datos:
 - Redimensionar imágenes a 64×64 px
 - Normalizar valores de píxel al rango [0, 1]
 - Dividir en train/validation/test con las proporciones provistas por el dataset
- Implementar una regresión lineal simple sobre una característica extraída manualmente (ej. intensidad media del píxel) y graficar la recta ajustada sobre los datos. Discutir por qué la regresión lineal no es suficiente para este problema.

Entregable de esta sección: gráficas del EDA, tabla de estadísticas y gráfica de regresión lineal, todo con celdas de texto explicativo.

SEMANA 2

Regresión logística como baseline

Temas que cubre: modelo matemático de la neurona, función sigmoide, regresión logística, entropía cruzada.

En esta sección el grupo debe:

- Implementar regresión logística desde cero (sin sklearn) usando las imágenes aplanadas como entrada:
 - Definir la función sigmoide $\sigma(z) = 1 / (1 + e^{(-z)})$
 - Implementar la función de pérdida de entropía cruzada binaria
 - Implementar el descenso de gradiente para actualizar pesos y sesgo
- Entrenar el modelo durante al menos 100 épocas y graficar la curva de pérdida.
- Reportar accuracy, precisión, recall y F1-score en el conjunto de test.
- Visualizar la matriz de confusión.
- Discutir: ¿qué limitaciones tiene la regresión logística para clasificar imágenes? ¿Qué aporta el modelo matemático de la neurona a esta formulación?

Este modelo servirá como baseline para comparar con los modelos más complejos de las secciones siguientes.



SEMANA 3

Red neuronal multicapa (MLP) con Backpropagation

Temas que cubre: redes multicapa, forward propagation, backpropagation, descenso de gradiente.

En esta sección el grupo debe:

- Implementar un MLP con al menos 2 capas ocultas usando el framework elegido (PyTorch o Keras):
 - Arquitectura sugerida: $64 \times 64 \rightarrow 512 \rightarrow 256 \rightarrow 1$ (salida con sigmoid)
 - Función de activación ReLU en capas ocultas
- Describir paso a paso la forward propagation para una imagen de ejemplo, mostrando las dimensiones de cada tensor en cada capa.
- Explicar en una celda Markdown el algoritmo de backpropagation: cómo fluyen los gradientes usando la regla de la cadena, cuál es el gradiente de la función de pérdida respecto a los pesos de la última capa.
- Entrenar el MLP y graficar curvas de pérdida y accuracy (train y validation) por época.
- Comparar los resultados del MLP contra el baseline de regresión logística (semana 2) en una tabla.

Celda Markdown obligatoria: derivar a mano (en LaTeX dentro del notebook) el gradiente $\partial L / \partial W$ para al menos una capa.

SEMANA 4

Regularización y optimizadores

Temas que cubre: overfitting, L1/L2, Dropout, Momentum, RMSProp, Adam.

En esta sección el grupo debe:

- Demostrar overfitting: entrenar el MLP de la sección anterior sin regularización con suficientes épocas hasta que la brecha train/val sea claramente visible en la gráfica.
- Aplicar tres técnicas de regularización por separado y comparar:
 - Regularización L2 (weight decay)
 - Dropout con $p = 0.5$ después de cada capa oculta
 - Early stopping basado en la pérdida de validación
- Comparar tres optimizadores sobre el MLP con las mismas condiciones iniciales:
 - SGD con Momentum
 - RMSProp
 - Adam



- Graficar las curvas de pérdida de los tres optimizadores en un mismo gráfico y concluir cuál converge más rápido y de forma más estable.
- Seleccionar la mejor combinación regularización + optimizador y justificar la elección. Este modelo será el MLP final de referencia.

Resultado esperado: tabla comparativa de las variantes con accuracy en validación y número de épocas hasta convergencia.

SEMANA 5

Convolución 2D y extracción de características

Temas que cubre: operador de convolución discreta, kernels, filtros, detección de bordes.

En esta sección el grupo debe:

- Implementar la operación de convolución 2D desde cero en NumPy (sin usar capas de PyTorch/Keras):
 - La función debe recibir una imagen y un kernel y retornar el mapa de características
 - Probar con imágenes del dataset
- Aplicar manualmente al menos tres kernels clásicos a una imagen de rayos X:
 - Kernel de detección de bordes horizontales (Sobel X)
 - Kernel de detección de bordes verticales (Sobel Y)
 - Kernel de suavizado (Gaussian blur 3×3)
- Visualizar la imagen original y los tres mapas de características resultantes en una sola figura con subplots.
- Explicar en una celda Markdown la conexión entre esta operación manual y la capa Conv2D de las redes neuronales: ¿qué aprende el modelo durante el entrenamiento que aquí se definió manualmente?
- Discutir cómo la convolución reduce el número de parámetros comparado con el MLP de la sección anterior para la misma imagen de entrada.

SEMANA 6

Arquitectura CNN con Batch Normalization y Transfer Learning

Temas que cubre: capas conv/pooling/FC, arquitecturas clásicas, Batch Normalization, Transfer Learning.



Esta sección tiene dos partes:

Parte A — CNN desde cero

- Diseñar e implementar una CNN inspirada en VGG con la siguiente estructura mínima:
 - Bloque 1: Conv(32) → BN → ReLU → MaxPool
 - Bloque 2: Conv(64) → BN → ReLU → MaxPool
 - Bloque 3: Conv(128) → BN → ReLU → MaxPool
 - Capas FC: 512 → 1 (sigmoid)
- Justificar en Markdown por qué se usa Batch Normalization: explicar el concepto de Internal Covariate Shift.
- Entrenar la CNN y comparar contra el MLP final (semana 4) en una tabla: accuracy, F1, tiempo de entrenamiento.

Parte B — Transfer Learning con ResNet-18

- Cargar ResNet-18 preentrenada en ImageNet (torchvision.models o tf.keras.applications).
- Aplicar la estrategia de Feature Extraction: congelar todas las capas convolucionales y reemplazar solo la capa FC final.
- Entrenar únicamente la capa final durante 10 épocas y reportar métricas.
- Comparar en una tabla final los cuatro modelos del trabajo: Regresión Logística, MLP, CNN desde cero, ResNet-18 Transfer Learning.

SEMANA
7

Evaluación completa del sistema y análisis de resultados

Temas que cubre: clasificación de imágenes, métricas de evaluación, accuracy, precision, recall, F1.

En esta sección el grupo debe:

- Evaluar el mejor modelo del trabajo completo sobre el conjunto de test (nunca visto durante el entrenamiento):
 - Accuracy, Precision, Recall, F1-score
 - Matriz de confusión
 - Curva ROC y valor AUC
- Visualizar al menos 5 predicciones correctas y 5 incorrectas del modelo, mostrando la imagen, la etiqueta real y la predicción con su probabilidad.
- Análisis de errores: ¿qué tienen en común las imágenes mal clasificadas? ¿Hay algún patrón?
- Tabla resumen comparativa de todos los modelos entrenados en el trabajo:



Modelo	Accuracy	Precision	Recall	F1-Score	Params
Regresión Logística (S2)	—	—	—	—	—
MLP + Regularización (S4)	—	—	—	—	—
CNN desde cero (S6A)	—	—	—	—	—
ResNet-18 Transfer (S6B)	—	—	—	—	—

- Reflexión final (celda Markdown): ¿qué aprendió el grupo sobre el proceso de diseño y entrenamiento de modelos de Deep Learning? ¿Qué cambiarían si tuvieran más tiempo o recursos?

4. Requisitos del Notebook

- El notebook debe ejecutarse de principio a fin sin errores en Google Colab.
- La primera celda debe ser Markdown con: nombres del grupo, fecha, y una tabla resumen de los 4 modelos con sus métricas finales en test.
- Cada sección debe comenzar con una celda Markdown que explique qué se va a hacer y por qué, conectando con la teoría de la semana correspondiente.
- Todas las gráficas deben tener: título, ejes etiquetados con unidades y leyenda cuando corresponda.
- Fijar semilla al inicio del notebook:

```
import torch; torch.manual_seed(42) # PyTorch
import tensorflow as tf; tf.random.set_seed(42) # TensorFlow
```

- No se permite copiar código de tutoriales sin adaptarlo y explicarlo. Cada bloque de código debe ir acompañado de comentarios inline y una celda Markdown de explicación.
- El notebook debe subirse con todos los outputs visibles, sin limpiar las celdas antes de entregar.



5. Rúbrica de Evaluación

Semana / Tema	Qué se evalúa	Puntaje	Nota
S1 — EDA y formulación	EDA completo con gráficas, formulación matemática correcta, preprocesamiento implementado.	2 pts	
S2 — Regresión logística	Implementación desde cero correcta, métricas reportadas, comparación con baseline.	3 pts	
S3 — MLP y Backpropagation	Arquitectura funcional, derivación de gradiente en LaTeX, comparación contra S2.	3 pts	
S4 — Regularización y optimiz.	Demostración de overfitting, 3 regularizaciones comparadas, 3 optimizadores comparados.	3 pts	
S5 — Convolución manual	Implementación NumPy correcta, 3 kernels aplicados, visualización, explicación conceptual.	3 pts	
S6 — CNN y Transfer Learning	CNN con BN funcional, Transfer Learning implementado, tabla comparativa de 4 modelos.	4 pts	
S7 — Evaluación y análisis	Métricas completas en test, visualización de errores, análisis crítico, reflexión final.	2 pts	

Nota: se descontará 1 punto si el notebook no es ejecutable de principio a fin en Colab, y 0.5 puntos por cada sección sin celdas Markdown explicativas.

6. Restricciones

- La sección S2 (regresión logística) debe implementarse desde cero, sin sklearn ni capas de PyTorch/Keras.
- La sección S5 (convolución 2D) debe implementarse desde cero en NumPy, sin torch.nn.Conv2d ni capas equivalentes.
- Las secciones S3–S7 sí pueden usar el framework elegido libremente.
- No se permite el uso de AutoML, Hugging Face Trainer completo, o herramientas que automaticen el entrenamiento.
- Todos los experimentos deben usar el mismo dataset y las mismas particiones train/val/test a lo largo de todo el notebook.
- Se permite (y se recomienda) usar Google Colab Pro o la GPU gratuita de Colab para las secciones S6 y S7.



7. Referencias Obligatorias

El notebook debe incluir una celda Markdown final con las siguientes referencias en formato APA o IEEE:

- LeCun et al. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *ICLR 2015*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training. *ICML*.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org>

Consultas al foro del aula virtual. ¡Mucho éxito!